

INE5408-03208A/B (20252) - Estruturas de Dados

[Painel](#) / [Cursos](#) / [INE5408-03208A/B \(20252\)](#) / Prova Prática I / [Prova Prática I](#)

 Descrição

 [Visualizar envios](#)

 **Disponível a partir de:** quarta-feira, 15 out. 2025, 15:10

 **Data de entrega:** quarta-feira, 15 out. 2025, 17:40

 **Arquivos requeridos:** linked_list.h ( [Baixar](#))

 **Número máximo de arquivos:** 10

Tipo de trabalho:  Trabalho individual

Enunciado. A lista encadeada está disponível, mas cinco (5) métodos novos devem ser implementados:

(1) Inversão da lista, ou seja, o primeiro passa a ser o último, o segundo passa a ser o penúltimo, e assim por diante:

▷ `void invert();`

Exemplo:

- Lista original: [A, B, C, D, E]
- Lista invertida após a chamada do método: [E, D, C, B, A]

(2) Duplicação da lista, ou seja, os mesmos elementos acrescentados, na mesma ordem, no final:

▷ `void doubling();`

Exemplo:

- Lista original: [A, B, C]
- Lista duplicada após a chamada do método: [A, B, C, A, B, C]

(3) Criação de uma lista com dados das posições de início (`start`) até fim (`stop`), saltando com um passo (`step`):

▷ `LinkedList<T> slicing(int start, int stop, int step);`

Exemplo:

- Lista original: [A, B, C, D, E, F, G, H]
- Lista de saída com `start=1`, `stop=5`, `step=2`: [B, D, F]

(4) Um ponto de cruzamento (*crossover*) entre a lista original e uma lista de entrada (`list_in`), de mesmo **tamanho par**, exatamente **no meio**:

▷ `void crossover(LinkedList<T> *list_in);`



Exemplo:

- Lista original: [A, B, C, D, E, F]
- Lista de entrada `list_in`: [M, N, O, P, Q, R]
- Lista original após esta operação: [A, B, C, P, Q, R]
- Lista `list_in` após esta operação: [M, N, O, D, E, F]

(5) Criação de uma lista contendo outras duas listas: a primeira, correspondente aos dados em posições pares; a segunda, correspondente aos dados em posições ímpares:

▷ `LinkedList< LinkedList<T> * > halve();`

Exemplo:

- Lista original: [A, B, C, D, E]
- Multilista de saída: [[A, C, E] , [B, D]]

Arquivos requeridos

linked_list.h

```

1  ///  
2  
3  #include <cstdint> // std::size_t  
4  #include <stdexcept> // C++ exceptions  
5  
6  
7  namespace structures {  
8  
9  ///  
10 template<typename T>  
11 class LinkedList {  
12 public:  
13     LinkedList(); // construtor padrão  
14     ~LinkedList(); // destrutor  
15     void clear(); // limpar lista  
16     void push_back(const T& data); // inserir no fim  
17     void push_front(const T& data); // inserir no início  
18     void insert(const T& data, std::size_t index); // inserir na posição  
19     void insert_sorted(const T& data); // inserir em ordem  
20     T& at(std::size_t index); // acessar um elemento na posição index  
21     T pop(std::size_t index); // retirar da posição  
22     T pop_back(); // retirar do fim  
23     T pop_front(); // retirar do início  
24     void remove(const T& data); // remover específico  
25     bool empty() const; // lista vazia  
26     bool contains(const T& data) const; // contém  
27     std::size_t find(const T& data) const; // posição do dado  
28     std::size_t size() const; // tamanho da lista  
29  
30     ///  
31     // Prova prática - implementações necessárias:  
32  
33     // (1) inverter a lista, ou seja, o primeiro passa a ser o último,  
34     // o segundo passa a ser o penúltimo, e assim por diante:  
35     void invert();  
36  
37     // (2) duplicar a lista, ou seja, acrescentar os mesmos elementos,  
38     // na mesma ordem, no final:  
39     void doubling();  
40  
41     // (3) criar uma lista com dados das posições de  
42     // início (start) até fim (stop), saltando com um passo (step):  
43     LinkedList<T> slicing(int start, int stop, int step);  
44  
45     // (4) criar um ponto de cruzamento (crossover) entre a lista original  
46     // e uma lista de entrada (list_in), de mesmo tamanho par,  
47     // exatamente no meio:  
48     void crossover(structures::LinkedList<T> *list_in);  
49  
50     // (5) criar uma lista contendo outras duas listas:  
51     // a primeira, correspondente aos dados em posições pares  
52     // a segunda, correspondente aos dados em posições ímpares:  
53     LinkedList< LinkedList<T> * > halve();  
54  
55     ///  
56 private:  
57     class Node { // Elemento  
58     public:  
59         ///  
60         //! Construtor usando apenas o dado.  
61         explicit Node(const T& data):  
62             data_{data}  
63         {}  
64  
65         //! Construtor de um nodo completo.  
66         explicit Node(const T& data, Node* next):  
67             data_{data},  
68             next_{next}  
69         {}  
70  
71         //! Retorna o dado armazenado.  
72         T& data() {  
73             return data_;  
74         }  
75  
76         //! Retorna o dado armazenado.  
77         const T& data() const {  
78             return data_;  
79         }  
80  
81         //! Retorna ponteiro para próximo nodo.  
82         Node* next() { // getter: próximo  
83             return next_;  
84         }  
85  
86         //! Retorna ponteiro para o o próximo node.  
87         const Node* next() const { // getter const: próximo  
88             return next_;  
89         }  
90  
91         //! Altera o ponteiro para o próximo.  
92         void next(Node* node) { // setter: próximo  
93             next_ = node;  
94         }  
95  
96     private:  
97         T data_; // data_  
98         Node* next_{nullptr}; // next_  
99 };  
100  
101 //! Passa pelos nós até o último.  
102 Node* end() { // último nó da lista  
103     auto it = head;  


```

```

104         for (auto i = 1u; i < size(); ++i) {
105             it = it->next();
106         }
107         return it;
108     }
109
110     ///! Passa pelos nós até o anterior ao índice procurado.
111     Node* before_index(std::size_t index) { // nó anterior ao index
112         auto it = head;
113         for (auto i = 1u; i < index; ++i) {
114             it = it->next();
115         }
116         return it;
117     }
118
119     void insert(const T& data, Node* before); // inserir na posição polimórfico
120
121     Node* head{nullptr}; // head
122     std::size_t size_{0u}; // size_
123 };
124
125 } // namespace structures
126
127 //*****
128
129 ///! Inversão da lista
130 template<typename T>
131 void structures::LinkedList<T>::invert() {
132     // ...
133     // COLOQUE SEU CODIGO AQUI...
134     // ...
135 }
136
137 ///! Duplicação da lista
138 template<typename T>
139 void structures::LinkedList<T>::doubling() {
140     // ...
141     // COLOQUE SEU CODIGO AQUI...
142     // ...
143 }
144
145 ///! Fatiamento da lista
146 template<typename T>
147 structures::LinkedList<T> structures::LinkedList<T>::slicing(int start,
148                                                             int stop,
149                                                             int step) {
150     LinkedList<T> list_slice;
151     // ...
152     // COLOQUE SEU CODIGO AQUI...
153     // ...
154     return list_slice;
155 }
156
157 ///! Crossover entre a lista original e uma lista de entrada (list_in)
158 template<typename T>
159 void structures::LinkedList<T>::crossover(structures::LinkedList<T> *list_in) {
160     // ...
161     // COLOQUE SEU CODIGO AQUI...
162     // ...
163 }
164
165 ///! Divisão da lista em duas partes (elementos com índices pares e ímpares)
166 template<typename T>
167 structures::LinkedList<T> structures::LinkedList<T> * >
168     structures::LinkedList<T>::halve() {
169     LinkedList<T> list_half;
170     // ...
171     // COLOQUE SEU CODIGO AQUI...
172     // ...
173     return list_half;
174 }
175
176 //*****
177
178 ///! Construtor padrão
179 template<typename T>
180 structures::LinkedList<T>::LinkedList() {}
181
182 ///! Destrutor
183 template<typename T>
184 structures::LinkedList<T>::~~LinkedList() {
185     clear();
186 }
187
188 ///! Esvazia a lista.
189 template<typename T>
190 void structures::LinkedList<T>::clear() {
191     while (!empty())
192         pop_front();
193 }
194
195 ///! Inserção no fim da lista.
196 template<typename T>
197 void structures::LinkedList<T>::push_back(const T& data) {
198     insert(data, size_);
199 }
200
201 ///! Inserção no começo da lista.
202 template<typename T>
203 void structures::LinkedList<T>::push_front(const T& data) {
204     Node* new_node = new Node(data);
205     new_node->next = head;
206     head = new_node;
207     size_++;
208 }

```

```

207         if (new_node == nullptr)
208             throw std::out_of_range("Full list!");
209
210         new_node->next(head);
211         head = new_node;
212         size_++;
213     }
214
215     ///! Inserção em qualquer posição da lista.
216     template<typename T>
217     void structures::LinkedList<T>::insert(const T& data, std::size_t index) {
218         if (index > size_)
219             throw std::out_of_range("Invalid index!");
220
221         if (index == 0) {
222             push_front(data);
223         } else {
224             Node* new_node = new Node(data);
225             if (new_node == nullptr)
226                 throw std::out_of_range("Full list!");
227
228             Node* before = before_index(index);
229             Node* next = before->next();
230             new_node->next(next);
231             before->next(new_node);
232             size_++;
233         }
234     }
235
236     ///! Inserção em qualquer posição da lista recebendo um ponteiro para um Node.
237     template<typename T>
238     void structures::LinkedList<T>::insert(const T& data, Node* before) {
239         Node* new_node = new Node(data);
240         if (new_node == nullptr)
241             throw std::out_of_range("Full list!");
242
243         new_node->next(before->next());
244         before->next(new_node);
245         size_++;
246     }
247
248     ///! Inserção ordenada na lista.
249     template<typename T>
250     void structures::LinkedList<T>::insert_sorted(const T& data) {
251         if (empty()) {
252             push_front(data);
253         } else {
254             Node* current = head;
255             Node* before = head;
256             std::size_t position = size();
257             for (auto i = 0u; i < size(); ++i) {
258                 if (!(data > current->data())) {
259                     position = i;
260                     break;
261                 }
262                 before = current;
263                 current = current->next();
264             }
265             position == 0? push_front(data) : insert(data, before);
266         }
267     }
268
269     ///! Coleta o dado na posição da lista.
270     template<typename T>
271     T& structures::LinkedList<T>::at(std::size_t index) {
272         if (index >= size())
273             throw std::out_of_range("Invalid index or empty list!");
274
275         Node* current = index == 0? head : before_index(index)->next();
276         return current->data();
277     }
278
279     ///! Coleta o dado de uma posição específica, removendo-o da lista.
280     template<typename T>
281     T structures::LinkedList<T>::pop(std::size_t index) {
282         if (empty())
283             throw std::out_of_range("Empty list");
284         if (index >= size())
285             throw std::out_of_range("Invalid index!");
286
287         if (index == 0)
288             return pop_front();
289
290         Node* before_out = before_index(index);
291         Node* out = before_out->next();
292         T data = out->data();
293         before_out->next(out->next());
294         size_--;
295         delete out;
296         return data;
297     }
298
299     ///! Coleta o dado do final, removendo-o da lista
300     template<typename T>
301     T structures::LinkedList<T>::pop_back() {
302         return pop(size_ - 1u);
303     }
304
305     ///! Coleta o dado do início, removendo-o da lista
306     template<typename T>
307     T structures::LinkedList<T>::pop_front() {
308         if (empty())
309             throw std::out_of_range("Empty list!");
310
311         Node* out = head;
312         head = head->next();
313         delete out;
314         size_--;
315         return out->data();
316     }

```

```
310
311     auto out = head;
312     T data = out->data();
313     head = out->next();
314     size_--;
315     delete out;
316     return data;
317 }
318
319 ///! Remoção de um dado da lista.
320 template<typename T>
321 void structures::LinkedList<T>::remove(const T& data) {
322     pop(find(data));
323 }
324
325 ///! Testa se a lista está vazia.
326 template<typename T>
327 bool structures::LinkedList<T>::empty() const {
328     return size() == 0u;
329 }
330
331 ///! Testa se um dado está na lista.
332 template<typename T>
333 bool structures::LinkedList<T>::contains(const T& data) const {
334     return find(data) != size();
335 }
336
337 ///! Procura o índice do dado; se não encontrar, retorna o tamanho da lista.
338 template<typename T>
339 std::size_t structures::LinkedList<T>::find(const T& data) const {
340     std::size_t index = 0u;
341     Node* current = head;
342     while (index < size()) {
343         if (current->data() == data)
344             break;
345         current = current->next();
346         index++;
347     }
348     return index;
349 }
350
351 ///! Retorna o tamanho da lista.
352 template<typename T>
353 std::size_t structures::LinkedList<T>::size() const {
354     return size_;
355 }
356
```

✉ Contate o suporte do site 

[VPL](#)

Você acessou como Elisa Belquis de Assumpcao (24202220) (Sair)
INE5408-03208A/B (20252)

Moodle UFSC - Presencial